CAPSTONE · DAY 2

Backend & AI Integration

Build and test a RAG-powered backend — FastAPI, LLM APIs and vector search wired into one retrieval pipeline.

FastAPI | OpenAI / Claude | FAISS · Pinecone | Embeddings & Retrieval

Builds on 20 days of foundations · Goal: validate the retrieval pipeline end-to-end

WHERE WE ARE

Today's mission

Twenty days built the fundamentals. Day 1 framed the capstone. Today we stand up the backend brain that makes it intelligent.

01

Set up the backend

Scaffold a FastAPI service with typed routes and auto docs.

02

Integrate LLM APIs

Wire in OpenAI chat completions with safe key handling.

03

Stand up a vector DB

Choose FAISS or Pinecone and store your embeddings.

04

Embed & retrieve

Chunk, embed, index, and pull top-k relevant context.

05

Test the endpoints

Validate every route and the full retrieval pipeline.

06

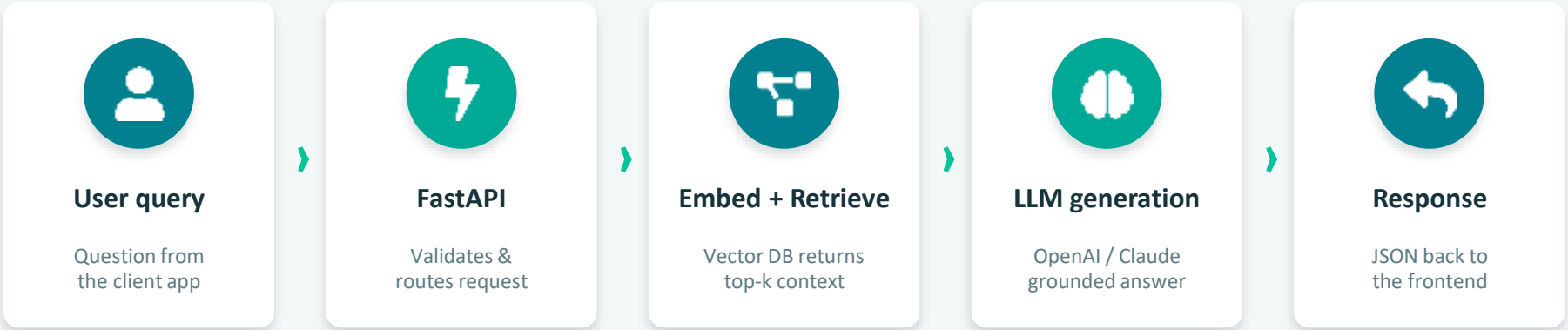
Model selection

Optional: route across Claude & OpenAI by cost & latency.

THE BIG PICTURE

Target architecture — a RAG pipeline

Every request flows through the same path: receive, embed, retrieve relevant context, then generate a grounded answer.



Why RAG? Grounding the model in your retrieved documents cuts hallucination and lets answers cite real, up-to-date sources.

STEP 1

Set up the FastAPI backend

FastAPI gives async performance, Pydantic validation, and interactive docs for free — ideal for an AI service.

Async by default

Handles slow LLM & DB calls without blocking.

Typed request models

Pydantic validates input before it hits your logic.

Auto-generated docs

Swagger UI at /docs the moment you run it.

main.py

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="RAG API")

class Query(BaseModel):
    question: str
    top_k: int = 4

@app.post("/ask")
def ask(q: Query):
    ctx = retrieve(q.question, q.top_k)
    ans = generate(q.question, ctx)
    return {"answer": ans,
            "sources": ctx}

# uvicorn main:app --reload
```

STEP 2

Integrate the LLM API (OpenAI)

Keep keys in environment variables, never in code. Tune the model, temperature, and token budget per route.

Secure the key

Load from .env — never commit secrets to git.

Pick model & params

model, temperature, max_tokens shape cost & tone.

Handle failures

Wrap calls with retries & timeouts for reliability.

llm.py

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["OPENAI_API_KEY"])

def generate(question, context):
    msg = f"Context:\n{context}\n\nQ:
{question}"
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0.2,
        messages=[{"role": "user",
                   "content": msg}],
    )
    return r.choices[0].message.content
```

STEP 3

Stand up a vector DB — FAISS vs Pinecone

Both store embeddings for similarity search. Choose by where you are in the project, not by hype.

FAISS

Local · open-source · free

Runs where? In your own process / on disk

Best for Prototyping & small–mid datasets

Scaling You manage memory & persistence

Cost Free; just compute you already have

Pinecone

Managed · cloud · scalable

Runs where? Fully hosted, accessed via API

Best for Production & large, growing data

Scaling Auto-scales, low-latency at volume

Cost Usage-based subscription

Rule of thumb: start on FAISS today to iterate fast, then graduate to Pinecone when data and traffic grow.

STEP 4

Embedding & retrieval logic

Turn text into vectors once, store them, then find the closest matches to each new question.

Chunk the docs

Split into overlapping passages the model can use.

Embed once

Convert chunks to vectors and index them.

Query top-k

Embed the question, return nearest neighbours.

retrieve.py

```
def embed(texts):
    r = client.embeddings.create(
        model="text-embedding-3-small",
        input=texts)
    return [d.embedding for d in r.data]

# index built once at startup
index.add(embed(chunks))

def retrieve(question, k=4):
    qv = embed([question])
    scores, ids = index.search(qv, k)
    return [chunks[i] for i in ids[0]]
```

END TO END

Anatomy of a single /ask request

Trace one question through the stack so you know exactly where to look when something breaks.

1

Receive & validate

FastAPI parses JSON into a Pydantic Query model and rejects bad input.

2

Embed the question

The query string is converted to a vector with the embedding model.

3

Retrieve context

The vector DB returns the top-k most similar document chunks.

4

Build the prompt

Retrieved chunks are stitched into the LLM prompt as grounding context.

5

Generate the answer

OpenAI (or Claude) returns a grounded completion.

6

Return JSON

Answer plus its source chunks are sent back to the frontend.

STEP 5

Test the API endpoints

Validate each route in isolation, then prove the whole retrieval pipeline returns grounded answers.

Swagger /docs

Hit every endpoint interactively the moment the server is up.

pytest + httpx

Automated tests for status codes, schemas, and edge cases.

curl / Postman

Quick manual checks and shareable request collections.

Pipeline check

Ask a known question — confirm correct, cited context returns.

Validate first: 200s on valid input, 422 on bad input, non-empty sources, and answers that match the retrieved context.

OPTIONAL · ADVANCED

Claude alongside OpenAI — model selection

Don't marry one model. Route each request to the best fit across three axes: cost, latency, and reasoning depth.

Cost

Cheap, high-volume calls → small / mini models. Reserve premium models for hard queries.

Latency

User-facing & real-time → fastest model. Background jobs can tolerate slower, deeper ones.

Reasoning

Multi-step logic, long context or nuance → route to the strongest reasoning model.

A simple routing strategy

Classify the request → cheap/fast model for simple FAQs, strong reasoning model for complex or high-stakes answers. *Log cost & latency per model so the routing rules improve with real data.*

HANDS-ON

Today's lab & definition of done

Build

- ✓ Scaffold a FastAPI app with /ask + /health routes
- ✓ Wire in the OpenAI client with keys from .env
- ✓ Index a small document set into FAISS
- ✓ Implement embed → retrieve → generate logic
- ✓ Return answers with their source chunks

Definition of done

- ✓ Server runs and /docs loads cleanly
- ✓ Endpoints pass status & schema tests
- ✓ Retrieval returns relevant, non-empty context
- ✓ An end-to-end question returns a grounded answer
- ✓ Secrets stay out of source control

DAY 2 — WRAP UP

You shipped the backend brain

FastAPI service

Typed, async, self-documenting.

LLM integrated

OpenAI wired in securely.

Vector retrieval

FAISS / Pinecone returning context.

Validated

Endpoints & pipeline tested.

Next up — Day 3: connect this backend to the frontend and harden the pipeline for real users.